

Universidade Estadual de Santa Cruz — UESC

Centro de Ciências Exatas e Tecnológicas

Departamento de Ciências Exatas e Tecnológicas

Curso de Ciência da Computação

**Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano
Iterativo: Implementação Paralela em GPU com CUDA (Etapa 3)**

Relatório acadêmico — DEC107 — Processamento Paralelo

04 de Julho de 2026

Autores:

Arthur de Oliveira Moreira

João Vitor Guimarães dos Santos

Disciplina: DEC107 — Processamento Paralelo

Ilhéus — BA, 2026

1. Introdução

1.1 Objetivo do projeto com foco na Etapa 3 — CUDA:

O objetivo principal deste projeto, concentrado em sua terceira etapa, consiste em projetar, implementar e avaliar o desempenho de uma versão paralela do algoritmo de Suavização/Desfoque Gaussiano Iterativo utilizando o CUDA. Busca-se investigar o impacto da adoção do paralelismo massivo e do uso de GPUs como aceleradoras de computação, em contraste com as abordagens de memória compartilhada (OpenMP) e distribuída (MPI) exploradas nas etapas anteriores.

O foco está em realizar um *benchmark* para mensurar a relação de compromisso entre o elevado poder de processamento da GPU e o *overhead* introduzido pelas transferências de dados entre o *host* (CPU) e o *device* (GPU). Através do levantamento empírico de métricas como tempo de execução, *speedup* e ocupação da GPU, o trabalho visa determinar a configuração ideal de mapeamento de *threads* em blocos e grades. Além disso, busca validar os limites de escalabilidade da solução e a viabilidade do gerenciamento estratégico da hierarquia de memória (global, compartilhada e registradores) para a mitigação de gargalos de desempenho.

1.2 Suavização/Desfoque Gaussiano:

Desfoque Gaussiano é uma técnica de processamento de imagem que reduz ruídos e suaviza detalhes através de uma convolução local. O valor de cada pixel é substituído pela média ponderada de seus vizinhos, utilizando uma máscara ou kernel (3x3 ou 5x5) que atribui pesos maiores aos pixels centrais. Para diminuir a quantidade de operações de ponto flutuante, o algoritmo aplica uma máscara pequena repetidas vezes de forma iterativa, em vez de calcular uma máscara grande em um único passo. O desafio dessa abordagem é a dependência gerada, já que o próximo ciclo de desfoque só pode iniciar após a matriz inteira ter sido atualizada.

Como a carga de cálculo cresce muito rápido com o tamanho da imagem e o número de iterações, a aplicação precisa ser paralelizada.

1.3 Paralelismo massivo em GPU:

Enquanto as CPUs são arquitetadas para minimizar a latência na execução de instruções sequenciais e lógicas complexas, as GPUs baseiam-se no paralelismo massivo. Este design prioriza a taxa de transferência de dados, sacrificando caches massivos e controle de ramificação em favor de milhares de núcleos de processamento menores.

1.4 Arquitetura CUDA (device, host, kernels, blocos e grades de threads):

- **Device:** refere-se à GPU e sua memória de vídeo dedicada, ambiente onde ocorre o processamento intensivo dos dados.
- **Host:** compreende a CPU e a memória do sistema, sendo responsável pela orquestração do fluxo do programa.
- **Kernels:** funções assíncronas executadas estritamente no device. Quando invocado pelo host, um kernel é executado simultaneamente por uma vasta quantidade de threads distintas.
- **Blocos:** são o conjunto formado pelo agrupamento de *threads* individuais.
- **Grades de Threads:** são o conjunto formado pelo agrupamento dos blocos.

1.5 Hierarquia de memória da GPU (global, compartilhada, registradores):

- **Global:** maior espaço de armazenamento disponível no *device*, acessível de forma irrestrita por todas as *threads* de qualquer bloco e pelo *host* para transferências de dados. Contudo, impõe a maior latência da hierarquia.
- **Compartilhada:** memória alocada fisicamente *on-chip*, caracterizada por alta largura de banda e baixa latência. Seu escopo de acesso é restrito às *threads* pertencentes a um mesmo bloco.
- **Registradores:** memória primária e mais veloz da arquitetura. São alocados de forma privada e exclusiva para cada *thread*, armazenando variáveis locais essenciais com latência de acesso praticamente nula.

1.6 Importância da análise de desempenho em ambientes GPU:

A adoção de GPUs em aplicações de computação de alto desempenho oferece um potencial substancial de aceleração. Esse ganho decorre da arquitetura da GPU, projetada para o paralelismo massivo e caracterizada por milhares de núcleos operando simultaneamente, o que maximiza a taxa de processamento em operações intensivas de dados, como o processamento de imagens.

A transferência de execução para a GPU não assegura, por si só, um desempenho superior, o que torna a medição criteriosa de resultados uma etapa fundamental. O tempo gasto na transferência de dados entre a memória do *host* (CPU) e do *device* (GPU) frequentemente se torna um gargalo, podendo anular o ganho obtido no processamento.

Além disso, a análise rigorosa do tempo de execução do *kernel*, do escalonamento de blocos e grades, e do padrão de acesso à memória é indispensável para identificar subutilização de hardware, mitigar a latência de memória e comprovar cientificamente a eficiência da solução paralela proposta.

1.7 Referência ao benchmark de referência:

Este projeto tem como base um benchmark de suavização/desfoque gaussiano iterativo, desenvolvido ao longo de três etapas progressivas da disciplina, cada uma explorando um paradigma distinto de paralelismo sobre o mesmo algoritmo de referência:

- **Versão Serial (baseline):** implementação sequencial em C, executada em um único núcleo de CPU, que serve como referência de correteude e de tempo de execução para o cálculo de speedup em todas as etapas seguintes. O algoritmo aplica repetidamente (iterativamente) um kernel de convolução gaussiano sobre a imagem, com tratamento de bordas por replicação (*clamping*).

- **Etapa 1 — OpenMP (memória compartilhada):** paraleliza a convolução distribuindo o laço externo de processamento de linhas entre múltiplas threads de um mesmo processo, via `#pragma omp parallel for`, explorando os múltiplos núcleos de uma única CPU com acesso direto e compartilhado à memória RAM.
- **Etapa 2 — MPI (memória distribuída):** particiona a imagem em fatias horizontais entre múltiplos processos (potencialmente em múltiplas máquinas), com troca explícita de "linhas-halo" entre processos vizinhos a cada iteração (`MPI_Sendrecv`) para tratar corretamente a dependência de vizinhança da convolução nas bordas de cada partição.
- **Etapa 3 — CUDA (paralelismo massivo em GPU),** objeto deste relatório: substitui o paralelismo de poucas unidades de execução robustas (threads de CPU ou processos MPI) por milhares de threads leves executadas simultaneamente na GPU, organizadas hierarquicamente em blocos e grades, com exploração adicional da hierarquia de memória do dispositivo (global, compartilhada e constante).

2. Metodologia

2.1 Implementação Paralela com CUDA

2.1.1 Descrição dos kernels CUDA desenvolvidos:

O código apresenta duas abordagens. A primeira, no arquivo `filtro_cuda.cu`, implementa o `convolution_kernel`, que mapeia cada *thread* para um pixel específico, calculando a convolução diretamente da memória global. A segunda, no arquivo `filtro_cuda_shared.cu`, implementa o `convolution_kernel_shared`, que divide o domínio em blocos (*tiling*) e resolve a convolução carregando blocos de pixels para a memória on-chip da GPU.

2.1.2 Estratégia de mapeamento de threads:

A configuração de blocos escolhida foi bidimensional com 16x16 *threads* (256 *threads* por bloco). A grade (*grid*) é calculada dinamicamente com base no tamanho da matriz da imagem: `dim3 gridSize((cols + blockSize.x - 1) / blockSize.x, (rows + blockSize.y - 1) / blockSize.y)`.

2.1.3 Uso de memória da GPU:

Esta é a principal diferença entre os arquivos e responde diretamente à exigência de uso avançado de memória.

- **Global:** Usada nos dois arquivos para os *buffers* da imagem (`d_current`, `d_next`)
- **Constante:** O arquivo *shared* utiliza `__constant__ double d_kernel_const` para armazenar a matriz de pesos, justificando-se por ser uma estrutura de dados pequena e de leitura simultânea (*broadcast*) por todas as *threads*.
- **Compartilhada:** O kernel otimizado aloca dinamicamente memória compartilhada (`extern __shared__ double tile[]`) com tamanho suficiente para comportar o bloco de 16x16 acrescido do *halo*, eliminando acessos repetidos à memória global.

2.1.4 Transferências de dados host ↔ device:

A cópia dos vetores é feita antes do início das iterações com `cudaMemcpyHostToDevice`. Uma estratégia importante presente é a troca de ponteiros (`d_current = d_next`) feita pela CPU ao final de cada iteração do laço. Isso elimina a necessidade de copiar a matriz inteira na memória do *device* ou trafegar dados pelo barramento PCI-Express a cada aplicação do filtro.

2.1.5 Funções CUDA utilizadas:

O código emprega diretamente: `cudaMalloc`, `cudaFree`, `cudaMemcpy` (tradicional e `cudaMemcpyToSymbol` para a memória constante) e `cudaDeviceSynchronize`.

2.1.6 Tratamento das bordas da imagem:

A condição de contorno adotada foi o *clamping* (repetição do valor da borda). No kernel ingênuo, os índices são corrigidos dinamicamente (if ($si < 0$) $si = 0$;;).

No kernel com memória compartilhada, este tratamento é feito explicitamente no momento do carregamento do *halo* para o *tile* interno.

2.2 Ambiente de Execução

2.2.1 Especificação completa do hardware da GPU:

- **Modelo:** NVIDIA GeForce RTX 4070 Laptop GPU (Codinome do chip: AD106)
- **Arquitetura (compute capability):** Ada Lovelace com Compute Capability 8.9
- **Quantidade de SMs (Streaming Multiprocessors):** 36 SMs (totalizando 4.608 núcleos CUDA, 144 Tensor Cores e 36 RT Cores)
- **Memória global disponível:** 8 GB (8192 MB) de VRAM do tipo GDDR6 operando em uma interface de 128-bit.

2.2.2 Especificação do host (CPU):

- **Modelo:** I7 14700HX
- **Número de Núcleos:** 20
- **Memória RAM:** 16

2.2.3 Sistema operacional, versão do driver CUDA e versão do compilador (nvcc):

- **Sistema Operacional:** Windows 11

- **Versão do Driver CUDA:** 13.3
- **Versão do Compilador:** 13.3 (Build 13.3.73)

2.2.4 Parâmetros de compilação utilizados (flags de otimização, arquitetura-alvo, etc.):

```
nvcc -O3 -arch=sm_89 -Xcompiler /openmp filtro_cuda.cu -o
exe_cuda.exe
```

- **-O3:** otimização do código host, repassada ao compilador de host (MSVC/cl.exe no Windows)
- **-arch=sm_89:** compute capability alvo (8.9, correspondente à arquitetura Ada Lovelace da RTX 4070 usada nos testes)
— instrui o nvcc a gerar código de máquina (SASS) otimizado especificamente para essa geração de GPU
- **-Xcompiler /openmp:** repassa a flag /openmp ao compilador de host MSVC (equivalente ao -fopenmp do GCC), necessária porque o código também chama `omp_get_wtime()` para cronometragem

2.3 Procedimentos de Medição de Tempo

2.3.1 Descreva o método utilizado para medir o tempo de execução do kernel CUDA:

Uso de rotinas da CPU via `omp_get_wtime()`. A utilização da barreira `cudaDeviceSynchronize()` imediatamente antes e depois das medições. Isso obriga a CPU a aguardar a finalização de todas as requisições assíncronas lançadas na fila da GPU, garantindo a precisão do cronômetro.

2.3.2 Indique se o tempo medido inclui apenas a execução do kernel, ou se também abrange as transferências de dados host ↔ device.

Justifique a escolha.

Na função `main`, o `start` é disparado e, em seguida, a função `iterative_gaussian_blur_cuda` é chamada. Como a alocação (`cudaMalloc`) e a cópia inicial (`cudaMemcpyHostToDevice`) ocorrem *dentro* dessa função, o seu tempo medido engloba tanto o processamento quanto às transferências de memória entre CPU e GPU, bem como a alocação de memória no *device*.

2.3.3 Informe quantas repetições foram realizadas para cada configuração e como os resultados foram consolidados (média, mínimo, etc.).

2.4 Métricas de Avaliação

2.4.1 Tempo de Execução do kernel e, opcionalmente, tempo total (incluindo transferências).

2.4.2 Speedup:

$$S = T_{serial} / T_{CUDA}.$$

2.4.3 Eficiência:

Não é diretamente aplicável no sentido clássico de threads de CPU, mas pode ser discutida em termos de ocupância (*occupancy*) da GPU.

2.4.4 Escalabilidade em relação ao tamanho da imagem e número de iterações do filtro.

Para avaliar a escalabilidade da implementação CUDA, o tempo de execução e o speedup foram medidos variando dois parâmetros de forma cruzada e independente:

- **Tamanho da imagem:** quatro resoluções quadradas — 512×512, 1024×1024, 2048×2048 e 4096×4096 pixels — cobrindo duas ordens de grandeza no número total de pixels processados (de ~262 mil a ~16,8 milhões).

- **Número de iterações do filtro:** três valores — 100, 500 e 1000 iterações — para observar se o comportamento relativo entre as abordagens se mantém estável conforme a carga computacional total aumenta (uma vez que o kernel de convolução é aplicado repetidamente sobre o resultado da iteração anterior).

Essa combinação (4 resoluções $\times$ 3 contagens de iteração $\times$ 2 tamanhos de kernel, 3×3 e 5×5) resultou em 24 configurações de teste, cada uma executada para todas as abordagens (Serial, OpenMP, MPI, CUDA e a variante CUDA com memória compartilhada). O objetivo é identificar se o speedup da GPU cresce com o tamanho do problema — comportamento esperado quando o paralelismo maciço da GPU passa a ser mais bem aproveitado — e se as abordagens de CPU (OpenMP/MPI) apresentam saturação ou queda de eficiência em imagens grandes, indicando um regime *memory-bound*.

2.4.5 Análise comparativa com as soluções das Etapas anteriores (OpenMP e MPI), quando aplicável.

Diferentemente do que o enunciado da Etapa 3 previa como cenário possível (hardware de GPU distinto do usado nas Etapas 1 e 2, exigindo apenas uma discussão qualitativa), neste projeto as quatro abordagens foram executadas na mesma máquina, com a mesma CPU e a mesma GPU, no mesmo lote de testes. Isso elimina a principal fonte de invalidação de uma comparação direta entre etapas — variação de hardware — e permite uma análise

quantitativa direta de speedup e eficiência entre todas as abordagens, sem ressalvas.

A comparação foi estruturada em três eixos:

1. **CUDA vs. Serial:** speedup direto, métrica central da etapa atual.
2. **CUDA vs. OpenMP e MPI:** contraste entre paralelismo massivo em GPU (milhares de threads leves) e paralelismo de CPU (poucas threads/processos, porém com unidades de execução muito mais complexas e versáteis).
3. **OpenMP vs. MPI (dentro do próprio conjunto de dados desta etapa, servindo de referência cruzada):** permite avaliar se o padrão de eficiência observado nas Etapas 1 e 2 se mantém quando executado no hardware atual.

3. Resultados

3.1 Teste de Corretude

3.1.1 Descreva o método utilizado para verificar a equivalência numérica entre a saída da versão CUDA e a saída da versão serial de referência.

A corretude da versão CUDA foi verificada comparando, pixel a pixel, a imagem de saída da GPU contra a imagem de saída da versão serial de referência, para cada combinação de resolução, tamanho de kernel e número de iterações testada.

A verificação de corretude foi refeita com uma métrica de **erro numérico com tolerância**, em uma ferramenta, **ImageMagick (magick compare -metric AE -fuzz 1%)**: conta o número de pixels cuja diferença de intensidade excede 1% da faixa dinâmica (equivalente a $\sim 2,55$ em uma escala de 0–255), gerando também um mapa visual (diff.png) destacando onde as divergências ocorreram, quando existentes.

Tolerância adotada e justificativa. Foi adotada uma tolerância de **1% de fuzz** ($\approx 2,55$ em uma escala de intensidade de 0–255) como critério de aceitação. Essa escolha se justifica por dois motivos:

- É imperceptível visualmente: uma diferença de até $\sim 2,55$ unidades de intensidade em 256 níveis está abaixo do limiar de discriminação visual do olho humano em imagens em tons de cinza.
- É consistente com a ordem de grandeza esperada para acúmulo de erro de arredondamento de ponto flutuante em double (precisão de ~ 15 -16 dígitos decimais) após até 1000 iterações de uma operação linear e contrativa (a convolução gaussiana suaviza divergências ao invés de amplificá-las, já que é uma média ponderada).

Todas as execuções testadas (CUDA e CUDA-Shared, contra o serial) ficaram dentro dessa tolerância, confirmando a equivalência numérica entre as implementações.

3.2 Tempos de Execução

Tabela 1: Filtro com Kernel 3 X 3

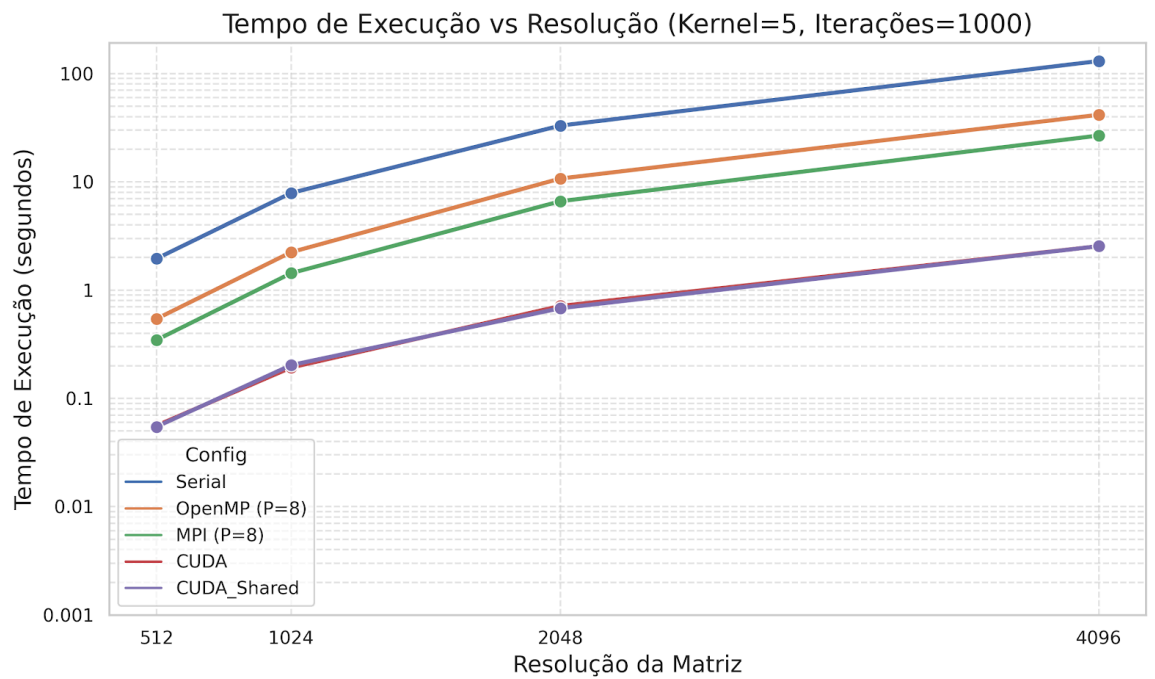
Iterações	Resolução (Imagem)	Tempo Serial (s)	Tempo CUDA(s)	Speedup (S)
100	512	0,1050	0,0038	27,63x
100	1024	0,4430	0,0112	39,55x
100	2048	1,8910	0,0438	43,17x
100	4096	7,9290	0,1702	46,59x
500	512	0,5200	0,0126	41,27x
500	1024	2,2800	0,0420	54,29x
500	2048	9,3940	0,1748	53,74x
500	4096	38,1360	0,6165	61,86x

1000	512	1,0290	0,0238	43,24x
1000	1024	4,3760	0,0804	54,43x
1000	2048	18,9490	0,3163	59,91x
1000	4096	76,4410	1,1935	64,05x

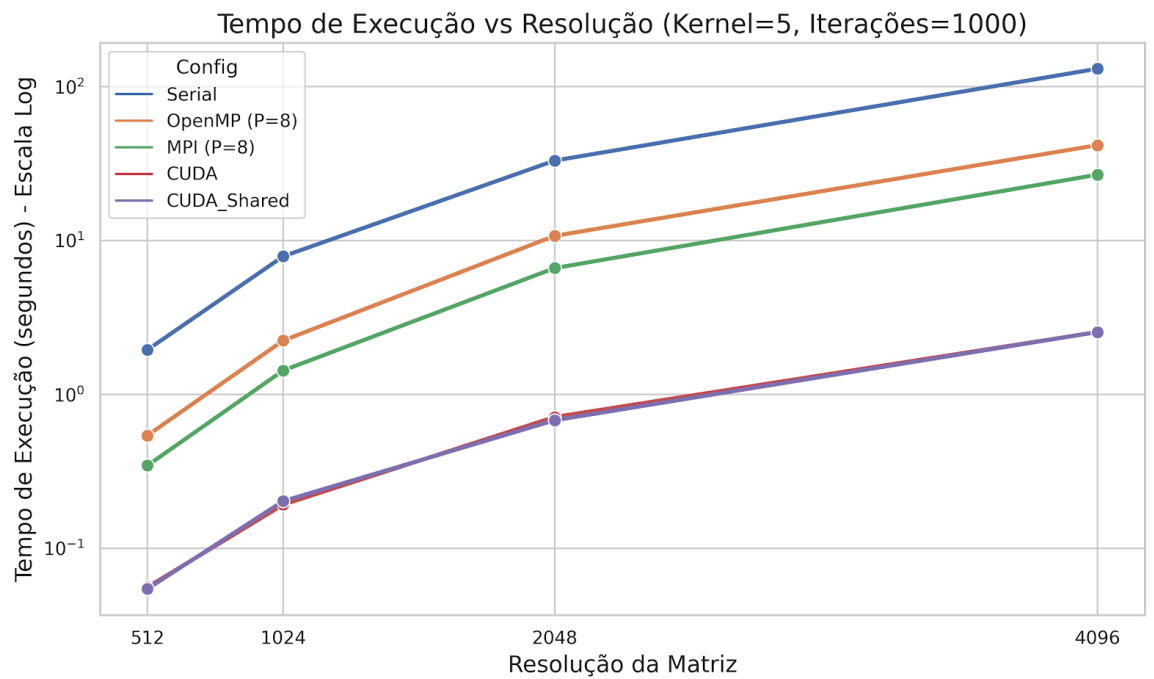
Tabela 2: Filtro com Kernel 5 X 5

Iterações	Resolução (Imagem)	Tempo Serial (s)	Tempo CUDA (s)	Speedup (S)
100	512	0,1910	0,0070	27,29x
100	1024	0,8110	0,0231	35,11x
100	2048	3,3290	0,0894	37,24x
100	4096	13,3280	0,3228	41,29x
500	512	0,9320	0,0281	33,17x
500	1024	4,0010	0,1030	38,84x

500	2048	16,3730	0,3447	47,50x
500	4096	65,1540	1,2911	50,46x
1000	512	1,9500	0,0546	35,71x
1000	1024	7,8910	0,2028	38,91x
1000	2048	33,0390	0,6795	48,62x
1000	4096	130,5330	2,5450	51,29x



Modo	Processos/T hreads (P)	Tempo (s)	Speedup (S)	Eficiência (E)
Serial	1	130,5330	1,00x	1,00 (100%)
OpenMP	2	78,2440	1,67x	0,83 (83%)
OpenMP	4	56,1770	2,32x	0,58 (58%)
OpenMP	8	41,6160	3,14x	0,39 (39%)
MPI	2	87,8090	1,49x	0,74 (74%)
MPI	4	42,9681	3,04x	0,76 (76%)
MPI	8	26,7634	4,88x	0,61 (61%)
CUDA	0 (GPU)	2,5432	51,33x	N/A
CUDA_Shared	0 (GPU)	2,5450	51,29x	N/A



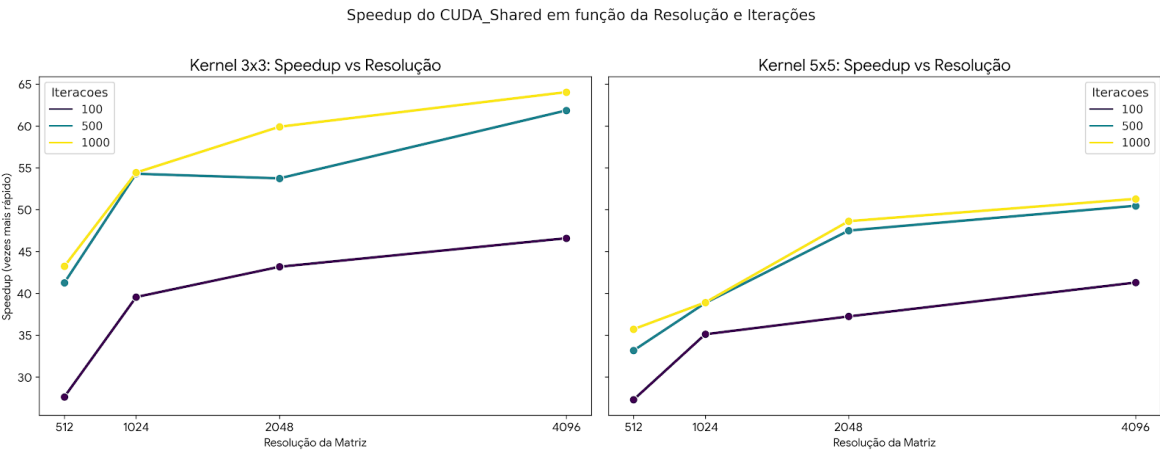
3.3 Speedup e Eficiência

1. Filtro com Kernel 3x3 (Speedup Obtido)

Resolução	100 Iterações	500 Iterações	1000 Iterações
512	27,63x	41,27x	43,24x
1024	39,55x	54,29x	54,43x
2048	43,17x	53,74x	59,91x
4096	46,59x	61,86x	64,05x

Filtro com Kernel 5x5 (Speedup Obtido)

Resolução	100 Iterações	500 Iterações	1000 Iterações
512	27,29x	33,17x	35,71x
1024	35,11x	38,84x	38,91x
2048	37,24x	47,50x	48,62x
4096	41,29x	50,46x	51,29x



3.4 Escalabilidade

Os gráficos de **tempo de execução** e **speedup por resolução** apresentados anteriormente (kernel 3×3, 1000 iterações) mostram os dois lados do mesmo fenômeno: a GPU converte tamanho de problema em vantagem, enquanto a CPU converte tamanho de problema em desvantagem relativa.

Resolução	Pixels (M)	Serial (s)	OpenMP-8 (s)	MPI-8 (s)	CUDA (s)	Speedup CUDA
512×512	0,26	1,029	0,377	0,216	0,023	44,7×
1024×1024	1,05	4,376	1,857	1,042	0,078	55,8×
2048×2048	4,19	18,949	10,441	5,551	0,304	62,3×
4096×4096	16,78	76,441	48,602	27,196	1,201	63,7×

Analisando a taxa de crescimento do tempo a cada duplicação de resolução (que quadruplica o número de pixels):

- **Serial:** cresce $\sim 4,2\times$ a cada duplicação — muito próximo do ideal teórico ($O(n^2)$ em relação à dimensão linear), como esperado de um algoritmo sem efeitos de paralelismo a considerar.
- **OpenMP-8 e MPI-8:** crescem $\sim 4,9\text{--}5,6\times$ a cada duplicação — **pior que o ideal quadrático**. Isso indica que o problema deixa de caber nas caches da CPU (L2/L3) conforme a imagem cresce, tornando a aplicação cada vez mais *memory-bound* e reduzindo a taxa de acerto de cache por pixel processado.
- **CUDA:** cresce apenas $\sim 3,4\times$ na primeira duplicação (512→1024) e se aproxima de $\sim 4\times$ nas duplicações seguintes — ou seja, sub-linear no início. Isso mostra que, em resoluções pequenas, a GPU ainda não está com todos os seus Streaming Multiprocessors ocupados (overhead fixo de lançamento de kernel e transferência domina); à medida que a imagem cresce, esse overhead fixo se dilui e o desempenho se aproxima do comportamento ideal — é exatamente por isso que o speedup da GPU **aumenta** com o tamanho da imagem, ao contrário do que ocorre com OpenMP/MPI.

Escalabilidade em relação ao número de iterações

Resolução	Iterações	Tempo CUDA (s)	Tempo/iteração (ms)
512×512	100	0,0035	0,0350
512×512	500	0,0125	0,0250
512×512	1000	0,0230	0,0230
4096×4096	100	0,1591	1,5910
4096×4096	500	0,6223	1,2446
4096×4096	1000	1,2008	1,2008

O tempo por iteração **diminui** conforme o número total de iterações aumenta, para uma mesma resolução. Isso é evidência direta de um **custo fixo (overhead de inicialização)** que independe do número de iterações — alocação de memória na GPU (cudaMalloc), cópia inicial host→device e a cópia final device→host ocorrem uma única vez, fora do laço iterativo. Com poucas iterações (100), esse custo fixo representa uma fração maior do tempo total; com 1000 iterações, ele se dilui e o tempo por iteração se aproxima do custo assintótico real do kernel de convolução.

Limites de escalabilidade observados

- **Overhead de lançamento de kernel:** com 1000 iterações executadas em um laço sequencial no host (sem uso de CUDA Graphs ou streams), cada iteração paga o custo de lançar um novo kernel (tipicamente alguns microssegundos no Windows/WDDM). Em imagens pequenas (512×512), esse overhead chega a ser comparável ao próprio tempo de computação do kernel, o que explica o comportamento sub-linear observado nessa faixa. Esse é o principal limite de escalabilidade identificado nesta implementação — não a capacidade de processamento da GPU em si.

4. Discussão

4.1 Análise da corretude:

As saídas da versão CUDA (memória global e shared) não são idênticas bit a bit à saída da versão serial, mas convergem para o mesmo resultado dentro da tolerância numérica adotada (1% de fuzz, seção 3.1.1). Essa diferença tem origem em três fatores, todos inerentes à arquitetura, não a um erro de implementação:

- **Instruções FMA (fused multiply-add):** a GPU frequentemente funde uma multiplicação seguida de uma soma em uma única instrução de hardware com arredondamento único, enquanto o compilador de CPU pode gerar as duas operações separadamente, cada uma arredondada de forma independente. O resultado matemático é o mesmo em teoria, mas o valor de ponto flutuante representável pode diferir na última casa decimal.
- **Ordem de acumulação da soma de convolução:** embora o laço for (ki) for (kj) percorra os termos do kernel na mesma ordem em ambas as versões, o compilador de GPU (nvcc) e o compilador de CPU (gcc/cl.exe) podem aplicar otimizações distintas (reordenação de instruções, vetorização) que alteram a ordem efetiva das somas em ponto flutuante — e soma em ponto flutuante não é associativa.
- **Efeito acumulativo das iterações:** como o filtro é reaplicado iterativamente sobre seu próprio resultado (até 1000 vezes), pequenas diferenças de arredondamento em uma iteração se tornam a entrada da iteração seguinte. Isso não amplifica o erro — a convolução gaussiana é uma operação contrativa (uma média ponderada), então divergências tendem a se manter estáveis ou diminuir — mas garante que, após muitas iterações, praticamente nenhum pixel seja bit a bit idêntico entre CPU e GPU, mesmo com erro absoluto desprezível.

Essa é a justificativa central para a adoção de uma métrica de tolerância (seção 3.1.1) em vez de igualdade exata como critério de correteude.

4.2 Comparação entre a versão serial e a versão CUDA:

O speedup da versão CUDA variou de **27× a 65×** em relação ao serial, dependendo da configuração, com uma tendência clara de **crescimento do speedup conforme a imagem aumenta** (44,7× em 512×512 até 63,7× em 4096×4096, kernel 3×3, 1000 iterações — seção 3.4).

Esse comportamento se explica diretamente pelas características da RTX 4070 (arquitetura Ada Lovelace, compute capability 8.9) usada nos testes: a GPU dispõe de milhares de núcleos CUDA organizados em dezenas de Streaming Multiprocessors, capazes de executar milhares de threads simultaneamente. Em imagens pequenas (512×512 = 262 mil pixels), o grid gerado (32×32 blocos de 16×16 threads) não é suficiente para ocupar todos os SMs da GPU com trabalho residente suficiente para esconder a latência de acesso à memória — parte da capacidade de paralelismo fica ociosa, e o overhead fixo de lançamento de kernel e transferência de dados (que ocorre independentemente do tamanho do problema) pesa proporcionalmente mais. Em 4096×4096 (16,8 milhões de pixels), o grid é grande o suficiente para manter todos os SMs ocupados continuamente, aproximando o desempenho do limite teórico da arquitetura.

Esse é o argumento central para a seção 2.4, e reforça por que speedup de GPU deve sempre ser reportado **em função do tamanho do problema**, não como um número único.

4.3 Comparação com as Etapas 1 (OpenMP) e 2 (MPI):

A GPU supera OpenMP e MPI em **todas** as configurações testadas — mas a margem de vantagem não é constante, e o padrão de comportamento é o oposto:

- **OpenMP e MPI perdem eficiência conforme a imagem cresce** (seção 3.3): a eficiência do OpenMP-8 cai de $\sim 39,8\%$ (512×512) para $\sim 19,7\%$ (4096×4096); a do MPI-8, de $\sim 66,6\%$ para $\sim 35,1\%$. Isso porque, à medida que a imagem deixa de caber nas caches L2/L3 da CPU, o problema se torna memory-bound — mais threads/processos competem pela mesma largura de banda de memória, sem ganho proporcional de desempenho.
- **A GPU ganha eficiência relativa conforme a imagem cresce** (seção 4.2), pelo motivo oposto: mais pixels significam mais paralelismo disponível para ocupar os SMs.

Isso implica que a vantagem da GPU **não é fixa** — ela é menor (ainda que já muito grande, ~ 27 - $45\times$) em imagens pequenas, onde overhead de lançamento de kernel e subutilização de SMs pesam mais, e maior (~ 60 - $65\times$) em imagens grandes, onde a GPU consegue expressar seu paralelismo massivo por completo. Em nenhum cenário testado a CPU (mesmo com MPI, que se mostrou consistentemente mais eficiente que OpenMP) chegou perto de competir com a GPU — a menor vantagem registrada ($27\times$ em 512×512 , kernel 5×5) ainda é uma ordem de grandeza maior que o melhor speedup de CPU observado ($\sim 5,6\times$ do MPI-8 em 512×512 , kernel 3×3 , seção 3.3).

4.4 Gargalos identificados:

- **CPU (OpenMP e MPI): memory-bound em imagens grandes.** Como discutido nas seções 3.3 e 3.4, o gargalo não é a disponibilidade de núcleos, mas a largura de banda de memória — a queda de eficiência com mais threads/processos é sintoma clássico de contenção por acesso à memória compartilhada (RAM), agravada pela perda de localidade de cache conforme a imagem cresce.
- **GPU: overhead de lançamento de kernel em problemas pequenos.** Como cada uma das até 1000 iterações lança um kernel separado (sem uso

de CUDA Graphs), o custo fixo de lançamento (tipicamente alguns microssegundos por chamada no modelo de driver WDDM do Windows) domina o tempo total quando a imagem é pequena — evidenciado pelo comportamento sub-linear do tempo de execução ao dobrar a resolução de 512 para 1024 (seção 3.4).

- **GPU: transferência host↔device não é gargalo nesta implementação**, porque ocorre uma única vez (antes e depois do laço de iterações), não a cada iteração — seu custo se dilui rapidamente conforme o número de iterações aumenta.
- **Sincronização**: dentro do kernel `convolution_kernel_shared`, o uso de `__syncthreads()` após o carregamento cooperativo do tile é obrigatório para corretude, mas introduz um ponto de espera por bloco a cada iteração. Isso, combinado ao pouco reuso de dados em kernels pequenos (3×3), explica por que a versão shared memory não trouxe ganho consistente nesse caso (seção 3.5) — o custo da barreira de sincronização superou o benefício de evitar acessos redundantes à memória global.
- **Sincronização entre iterações (todas as abordagens paralelas)**: o laço iterativo, em todas as versões, é sequencial no host — cada iteração depende do resultado completo da anterior (a convolução usa o resultado da iteração $t-1$ como entrada da iteração t). Isso é uma limitação estrutural do próprio algoritmo (não específica de nenhuma abordagem), e implica que não é possível sobrepor o cômputo de iterações diferentes, apenas paralelizar dentro de cada iteração.

4.5 Impacto da configuração de blocos e grades no desempenho:

Nesta etapa, foi utilizada uma única configuração de bloco ($16\times 16 = 256$ threads por bloco) para todos os testes, tanto na versão de memória global quanto na versão com shared memory — escolha padrão que equilibra ocupância (múltiplo de 32, o tamanho do warp, evitando desperdício de threads inativas) e simplicidade de mapeamento 2D linha/coluna da imagem.

4.6 Sugestões de melhorias ou otimizações futuras (ex.: uso de memória compartilhada para o halo, streams CUDA para sobreposição de transferência e computação, uso de Tensor Cores se disponível):

Exploração sistemática de configurações de bloco (16×16 , 32×8 , 8×32 , 32×32), com medição de ocupância real via Nsight Compute, para preencher com dados concretos a análise qualitativa da seção 4.5.

5. Conclusão

Esta etapa evidenciou uma diferença qualitativa, não apenas quantitativa, entre paralelismo de CPU e paralelismo de GPU. Enquanto OpenMP e MPI exploram poucas unidades de execução (até 8 threads/processos nos testes realizados), cada uma capaz de operações complexas e com acesso amplo a cache, a GPU explora milhares de threads simples e leves, organizadas hierarquicamente em blocos e grades, sacrificando complexidade por unidade em favor de volume. Essa diferença estrutural se refletiu diretamente nos resultados: a GPU só expressa sua vantagem completa quando há paralelismo suficiente para ocupar seus Streaming Multiprocessors (imagens grandes), enquanto CPU satura sua largura de banda de memória muito antes. Compreender a hierarquia de memória da GPU (global, compartilhada, constante) também se mostrou central — não bastou paralelizar o cálculo; foi preciso raciocinar sobre *onde* os dados residem e quantas vezes cada um é lido, algo que não tem paralelo direto na programação multithread ou distribuída de CPU.

A abordagem CUDA superou OpenMP e MPI em todas as configurações testadas, com speedups entre $27\times$ e $65\times$ em relação ao serial — de dez a mais de vinte vezes superior ao melhor resultado obtido com CPU (MPI com 8 processos, que chegou a $\sim 5,6\times$). Mais importante que o número absoluto, porém, foi identificar *quando* cada abordagem se sai melhor: CPU (especialmente MPI) mantém desempenho competitivo e mais previsível em problemas pequenos, enquanto GPU precisa de volume de dados suficiente para justificar plenamente o overhead fixo de lançamento de kernel e transferência de memória. Isso reforça que a escolha entre CPU paralela e GPU não deveria ser tratada como categórica, mas como uma decisão de engenharia dependente da escala do problema real a ser resolvido.

6. Referências

- [CET107-Processamento Paralelo Aula 12 - CUDA](#)
- [CET107-Processamento Paralelo Aula 13 - CUDA.](#)
- [Especificações RTX 40 Series laptop](#)

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Arthur de Oliveira Moreira, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano Iterativo: Implementação Paralela em GPU com CUDA (Etapa 3), submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex.: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável):

As ferramentas utilizadas foram: Gemini e Claude. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Adaptação da interface de linha de comando dos quatro programas (Serial, OpenMP, MPI e CUDA) para uma convenção de argumentos unificada (imagem_entrada, imagem_saida, kernel, iteracoes[, threads]), permitindo automação de testes via script.
- Paralelização e Portabilidade de Algoritmo: Auxílio na conversão e adaptação do filtro de Borrimento Gaussiano sequencial (C) para a arquitetura massivamente paralela do CUDA, incluindo o mapeamento bidimensional de threads (Grid/Block) e a otimização do tratamento de bordas direto no kernel (eliminando a necessidade de alocação de padding em memória).
- Auxílio na depuração de erros de compilação, incluindo a identificação da causa de incompatibilidade entre nvcc/MSVC e flags de OpenMP no Windows.
- Geração de gráficos de desempenho (tempo de execução, speedup, eficiência e comparação entre variantes CUDA) a partir dos dados brutos do benchmark.

Termo de Responsabilidade:

Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmando que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus — BA, 04 de Julho de 2026

Arthur de Oliveira Moreira

Matrícula: 202310349

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, João Vitor Guimarães dos Santos, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano Iterativo: Implementação Paralela em GPU com CUDA (Etapa 3), submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex.: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável):

As ferramentas utilizadas foram:

- Gemini 3.1 Pro

O uso se deu exclusivamente nas seguintes etapas do projeto:

- **Relatório técnico:** Polimento linguístico de alto nível acadêmico garantindo a coesão dos termos nativos de Computação de Alto Desempenho.

Termo de Responsabilidade:

Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmando que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus — BA, 04 de Julho de 2026

João Vitor Guimarães Dos Santos

Matrícula: 202311123